

XE DAP THONG NHAT — TNBike

Bao Cao Phan Tich Du Lieu — 8 Du An Khoa Hoc Du Lieu

1. Kham Pha & Lam Sach Du Lieu Ban Hang
2. Du Doan Doanh Thu — Ridge Regression (da sua data leakage)
3. Du Bao Chuoi Thoi Gian — AR/ARMA | **C.2 Cau 1**
4. Phan Loai Don Hang Gia Tri Cao
5. Phat Hien Churn Dai Ly — Random Forest | **C.2 Cau 3**
6. Phan Nhom Khach Hang RFM — KMeans + PCA
7. AB Testing Chien Dich Ban Hang
8. Du Bao Bien Dong Doanh Thu — GARCH

DB: tnbike_db (Neon Cloud) | 17,031 giao dich | 2025 Q1 - 2026 Q1

Xe Đạp Thống Nhất — Khám Phá Dữ Liệu Bán Hàng

Notebook 2: Làm Sạch Dữ Liệu

Xây dựng hàm wrangle() và làm sạch dữ liệu. Theo cấu trúc WQU ADSL Dự Án 1.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import matplotlib.pyplot as plt
import pandas as pd
from sqlalchemy import create_engine
```

1. Kết Nối Cơ Sở Dữ Liệu

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

2. Hàm wrangle() — Tải & Làm Sạch Dữ Liệu

In []:

```
def wrangle(engine):
    df = pd.read_sql_query(
        '''
        SELECT
        order_date, fiscal_year, fiscal_quarter, fiscal_month,
        so_number, customer_code, customer_name,
        province_name, region,
        product_name, color, line_name, group_code, group_name,
        quantity,
        ROUND(unit_price/1000000.0,2) AS unit_price,
        ROUND(line_total/1000000.0,2) AS line_total
        FROM tnbike.fact_sales
        WHERE order_date BETWEEN '2025-01-01' AND '2026-03-31'
        ''',
        con=engine
    )

    # Bỏ các dòng có giá trị thiếu
    df.dropna(inplace=True)

    # Loại bỏ ngoại lệ doanh thu (giữ 10% - 90%)
    thap, cao = df['line_total'].quantile([0.10, 0.90])
    df = df[df['line_total'].between(thap, cao)]

    # Chuyển đổi kiểu ngày
    df['order_date'] = pd.to_datetime(df['order_date'])

    return df

df = wrangle(engine)
print('Kích thước sau wrangle:', df.shape)
df.head()
```

3. Thống Kê Mô Tả

In []:

```
df[['quantity', 'unit_price', 'line_total']].describe()
```

4. Phân Phối line_total

In []:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(df['line_total'], bins=30)
axes[0].set_title('Biểu Đồ Tần Suất – Doanh Thu')
axes[0].set_xlabel('Doanh Thu (Triệu VNĐ)')
axes[1].boxplot(df['line_total'])
axes[1].set_title('Hộp Số – Doanh Thu')
```

```
axes[1].set_ylabel('Doanh Thu (Triệu VND)')
plt.tight_layout()
plt.show()
```

5. Doanh Thu Trung Bình Theo Nhóm Sản Phẩm

In []:

```
tb_nhom = df.groupby('group_name')['line_total'].mean().sort_values(ascending=True)
tb_nhom.plot(
    kind='bar',
    xlabel='Nhóm Sản Phẩm',
    ylabel='Doanh Thu TB (Triệu VND)',
    title='Doanh Thu Trung Bình Theo Nhóm Sản Phẩm'
)
plt.tight_layout()
plt.show()
```

6. Thống Kê Phân Loại

In []:

```
df['group_name'].value_counts()
```

In []:

```
df['region'].value_counts()
```

In []:

```
engine.dispose()
print('Đã đóng kết nối.')
```

Xe Đạp Thống Nhất — Dự Đoán Doanh Thu

Notebook 3: Mô Hình Hồi Quy Ridge

Xây dựng pipeline Ridge Regression để dự đoán doanh thu. Theo cấu trúc WQU ADSL Dự Án 2.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from category_encoders import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error
from sklearn.pipeline import make_pipeline
from sqlalchemy import create_engine
```

1. Tải Dữ Liệu

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
def wrangle(engine):
    df = pd.read_sql_query(
        '''
        SELECT
        fs.fiscal_month, fs.region, fs.group_name,
        fs.line_name, fs.color,
        fs.quantity, ROUND(fs.unit_price/1000000.0,2) AS unit_price, ROUND(fs.line_total/1000000.0,2) AS line_total
        FROM tnbike.fact_sales fs
        WHERE fs.order_date BETWEEN '2025-01-01' AND '2026-03-31'
        ''', con=engine
    )
    thap, cao = df['line_total'].quantile([0.10, 0.90])
    df = df[df['line_total'].between(thap, cao)]
    df.dropna(inplace=True)
    return df

df = wrangle(engine)
df.head()
```

2. Tách Tập Huấn Luyện / Kiểm Tra

In []:

```
from sklearn.model_selection import train_test_split

nhan = 'line_total'
# Bỏ quantity và unit_price để tránh data leakage
# (line_total = quantity * unit_price — không dùng thành phần tạo ra nhãn)
dac_trung = ['fiscal_month', 'group_name', 'line_name', 'color', 'region']

X = df[dac_trung]
y = df[nhan]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print('X_train:', X_train.shape, '| X_test:', X_test.shape)
```

3. Mô Hình Cơ Sở (Baseline)

In []:

```
y_trung_binh = y_train.mean()
y_du_doan_cb = [y_trung_binh] * len(y_train)
mae_cb = mean_absolute_error(y_train, y_du_doan_cb)
print('MAE Baseline (trung bình):', round(mae_cb, 4), 'Triệu VNĐ')
```

4. Xây Dựng Pipeline Ridge Regression

In []:

```
mo_hinh = make_pipeline(
    OneHotEncoder(use_cat_names=True),
    SimpleImputer(),
    Ridge()
)
mo_hinh.fit(X_train, y_train)
print('Đã huấn luyện mô hình.')
```

5. Đánh Giá Mô Hình

In []:

```
y_du_doan = pd.Series(mo_hinh.predict(X_test))
mae_mo_hinh = mean_absolute_error(y_test, y_du_doan)
print('MAE Baseline :', round(mae_cb, 4), 'Triệu VNĐ')
print('MAE Ridge Test :', round(mae_mo_hinh, 4), 'Triệu VNĐ')
print('Cải thiện :', round((mae_cb - mae_mo_hinh)/mae_cb*100, 1), '%')
```

6. Tầm Quan Trọng Đặc Trưng (Hệ Số Ridge)

In []:

```
he_so = mo_hinh.named_steps['ridge'].coef_
ten_dac_trung = mo_hinh.named_steps['onehotencoder'].get_feature_names_out()

quan_trong = pd.Series(he_so, index=ten_dac_trung).sort_values()
quan_trong.tail(10).plot(kind='barh')
plt.xlabel('Hệ Số Ridge')
plt.title('Top 10 Đặc Trưng Quan Trọng Nhất')
plt.tight_layout()
plt.show()
```

7. Kết Quả Dự Đoán

In []:

```
y_du_doan.head(10)
```

Câu Hỏi 2 (C.2): Dự Báo Màu Sắc & Cơ Cấu Q2/2026

Màu sắc nào tăng nhu cầu theo mùa? Cơ cấu màu sắc Q2/2026? SKU nào có nguy cơ bán chậm?

In []:

```
# Câu hỏi 2a: Phân tích xu hướng màu sắc theo quý
# fact_sales đã có sẵn color, quantity, line_total – không cần JOIN thêm
df_mau = pd.read_sql_query(
    '''
    SELECT
        fiscal_year AS nam,
        fiscal_quarter AS quy,
        color,
        SUM(quantity) AS tong_sl,
        ROUND(SUM(line_total)/1000000.0,2) AS tong_dt
    FROM tnbike.fact_sales
    WHERE color IS NOT NULL AND color <> ''
    GROUP BY fiscal_year, fiscal_quarter, color
    ORDER BY fiscal_year, fiscal_quarter, tong_sl DESC
    ''',
    engine
)
df_mau['ky'] = df_mau['nam'].astype(str) + '-Q' + df_mau['quy'].astype(str)
print('Dữ liệu màu theo quý:', df_mau.shape)
df_mau.head()
```

In []:

```
# Tỷ trọng màu sắc theo quý (pivot)
pivot_mau = df_mau.pivot_table(
    index='ky', columns='color', values='tong_sl', aggfunc='sum', fill_value=0
)
ty_trong_mau = pivot_mau.div(pivot_mau.sum(axis=1), axis=0) * 100

top_mau = ty_trong_mau.iloc[-4:].T.mean().nlargest(6).index.tolist()
print('Top 6 màu phổ biến:', top_mau)
ty_trong_mau[top_mau].tail(6)
```

In []:

```
# Biểu đồ xu hướng màu sắc qua các quý
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 5))
ty_trong_mau[top_mau].plot(ax=ax, marker='o')
ax.axvline(x=len(ty_trong_mau)-1, color='red', linestyle='--', alpha=0.5, label='Hiện tại')
ax.set_title('Tỷ Trọng Màu Sắc Theo Quý (%)', fontsize=14)
ax.set_xlabel('Quý')
ax.set_ylabel('Tỷ trọng (%)')
ax.legend(bbox_to_anchor=(1.01, 1))
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

In [ ]:

# Câu hỏi 2b: Dự báo cơ cấu màu sắc Q2/2026 (tháng 4-6)
# Dùng hệ số Q2/Q1 từ năm 2025 làm tỷ lệ mùa vụ

q1_2025_mau = df_mau[(df_mau['nam']==2025) & (df_mau['quy']==1)].set_index('color')['tong_sl']
q2_2025_mau = df_mau[(df_mau['nam']==2025) & (df_mau['quy']==2)].set_index('color')['tong_sl']
q1_2026_mau = df_mau[(df_mau['nam']==2026) & (df_mau['quy']==1)].set_index('color')['tong_sl']

he_so_mua_vu = (q2_2025_mau / q1_2025_mau).fillna(1.0).replace([float('inf'), -float('inf')], 1.0)

du_bao_q2_2026_mau = (q1_2026_mau * he_so_mua_vu).dropna().sort_values(ascending=False)
tong_du_bao = du_bao_q2_2026_mau.sum()
co_cau_du_bao = (du_bao_q2_2026_mau / tong_du_bao * 100).round(2)

print('=== Dự Báo Cơ Cấu Màu Sắc Q2/2026 ===')
print(co_cau_du_bao.to_string())
```

```
In [ ]:

# Biểu đồ cơ cấu màu sắc dự báo Q2/2026
fig, ax = plt.subplots(figsize=(10, 5))
co_cau_du_bao.head(8).sort_values().plot(kind='barh', ax=ax, color='steelblue')
ax.set_title('Dự Báo Cơ Cấu Màu Sắc Q2/2026 (%)', fontsize=13)
ax.set_xlabel('Tỷ trọng (%)')
for bar in ax.patches:
    ax.text(bar.get_width() + 0.2, bar.get_y() + bar.get_height()/2,
            f'{bar.get_width():.1f}%', va='center')
plt.tight_layout()
plt.show()
```

```
In [ ]:

# Câu hỏi 2c: SKU có nguy cơ bán chậm (nhu cầu giảm)
df_sku_mau = pd.read_sql_query(
    '''
    SELECT
    product_code,
    product_name,
    color,
    fiscal_year AS nam,
    fiscal_quarter AS quy,
    SUM(quantity) AS tong_sl
    FROM tnbike.fact_sales
    WHERE fiscal_year IN (2024, 2025, 2026)
    AND color IS NOT NULL AND color <> ''
    GROUP BY product_code, product_name, color, fiscal_year, fiscal_quarter
    ''',
    engine
)

# So sánh Q1/2026 vs Q1/2025
q1_25_sku = df_sku_mau[(df_sku_mau['nam']==2025)&(df_sku_mau['quy']==1)].set_index('product_code')['tong_sl']
q1_26_sku = df_sku_mau[(df_sku_mau['nam']==2026)&(df_sku_mau['quy']==1)].set_index('product_code')['tong_sl']

tang_truong = ((q1_26_sku - q1_25_sku) / q1_25_sku * 100).dropna().sort_values()
sku_giam = tang_truong[tang_truong < -10]

print(f'SKU có nguy cơ bán chậm (giảm >10% so Q1/2025): {len(sku_giam)}')
print(sku_giam.head(10).to_string())
```

```
In [ ]:

# Bảng tóm tắt SKU nguy cơ bán chậm kèm thông tin màu sắc
ten_sku = df_sku_mau[['product_code', 'product_name', 'color']].drop_duplicates().set_index('product_code')
df_nguy_co = pd.DataFrame({
    'tang_truong_pct': tang_truong,
    'sl_q1_2025': q1_25_sku,
    'sl_q1_2026': q1_26_sku
}).join(ten_sku).sort_values('tang_truong_pct')

df_nguy_co_hien = df_nguy_co[df_nguy_co['tang_truong_pct'] < -10].head(15)
print('=== Top 15 SKU Có Nguy Cơ Bán Chậm ===')
print(df_nguy_co_hien[['product_name', 'color', 'sl_q1_2025', 'sl_q1_2026', 'tang_truong_pct']].to_string())
```

Phương Trình Hồi Quy Ridge

$$\hat{y} = \beta_0 + \beta_1 X_{\text{quantity}} + \beta_2 X_{\text{unit_price}} + \beta_3 X_{\text{line_name}} + \dots + \beta_n X_n$$

β (beta) = hệ số ảnh hưởng | **X** = đặc trưng đầu vào | **ŷ** = doanh thu dự báo (Triệu VNĐ)

In []:

```
from IPython.display import display, HTML
import numpy as np

# Lấy hệ số và tên đặc trưng
he_so = mo_hinh.named_steps['ridge'].coef_
ten_dac_trung = mo_hinh.named_steps['onehotencoder'].get_feature_names_out()
df_beta = pd.DataFrame({'Đặc Trưng (X)': ten_dac_trung, 'Hệ Số β': he_so})
df_beta = df_beta.reindex(df_beta['Hệ Số β'].abs().sort_values(ascending=False).index)

# Hiển thị top 10 β với màu sắc
def to_html_row(row):
    color = '#2196F3' if row['Hệ Số β'] > 0 else '#F44336'
    return f"<tr><td style='color:goldenrod;font-weight:bold'>{row['Đặc Trưng (X)']}</td>\n"
    f"<td style='color:{color};font-weight:bold;text-align:right'>{row['Hệ Số β']:+.4f}</td></tr>"

html = '<table style="font-size:13px;border-collapse:collapse">'
html += '<tr><th style="padding:4px 12px;background:#f0f0f0">Đặc Trưng X</th>'
html += '<th style="padding:4px 12px;background:#f0f0f0">Hệ Số β</th></tr>'
for _, row in df_beta.head(10).iterrows():
    html += to_html_row(row)
html += '</table>'
display(HTML('<h4>Top 10 Hệ Số β Có Ảnh Hưởng Lớn Nhất</h4>' + html))
```

In []:

```
# Chỉ tiêu hiệu quả mô hình
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred_test = mo_hinh.predict(X_test)
mae = mean_absolute_error(y_test, y_pred_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2 = r2_score(y_test, y_pred_test)
mape = np.mean(np.abs((y_test - y_pred_test) / (y_test + 1e-9))) * 100

metrics_html = f'''
<div style="display:flex;gap:16px;flex-wrap:wrap;margin:12px 0">
  <div style="background:#E3F2FD;padding:12px 20px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">MAE (Triệu VNĐ)</div>
    <div style="font-size:22px;font-weight:bold;color:#1565C0">{mae:.3f}</div>
  </div>
  <div style="background:#FFF9C4;padding:12px 20px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">RMSE</div>
    <div style="font-size:22px;font-weight:bold;color:#F57F17">{rmse:.3f}</div>
  </div>
  <div style="background:#E8F5E9;padding:12px 20px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">R² Score</div>
    <div style="font-size:22px;font-weight:bold;color:#2E7D32">{r2:.4f}</div>
  </div>
  <div style="background:#FCE4EC;padding:12px 20px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">MAPE (%)</div>
    <div style="font-size:22px;font-weight:bold;color:#B71C1C">{mape:.1f}%</div>
  </div>
</div>'''
display(HTML('<h4>Chỉ Tiêu Hiệu Quả Mô Hình</h4>' + metrics_html))
```

Công Cụ Dự Báo Doanh Thu (Non-Tech)

Chọn thông tin sản phẩm → hệ thống tự tính doanh thu dự báo

In []:

```
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# Lấy giá trị unique từ dữ liệu
ds_nhom = sorted(df['group_name'].dropna().unique().tolist())
ds_vung = sorted(df['region'].dropna().unique().tolist())
ds_mau = sorted(df['color'].dropna().unique().tolist()) if 'color' in df.columns else ['Không rõ']

w_sl = widgets.IntSlider(value=2, min=1, max=20, description='Số lượng:', style={'description_width':'120px'})
w_gia = widgets.FloatSlider(value=1.3, min=0.5, max=8.0, step=0.05, description='Đơn giá (Tr):', style={'description_width':'120px'})
w_nhom = widgets.Dropdown(options=ds_nhom, description='Nhóm xe:', style={'description_width':'120px'})
w_vung = widgets.Dropdown(options=ds_vung, description='Vùng:', style={'description_width':'120px'})
w_thang = widgets.IntSlider(value=4, min=1, max=12, description='Tháng:', style={'description_width':'120px'})
out = widgets.Output()
```

```

def du_bao(_):
    with out:
        clear_output(wait=True)
        row = pd.DataFrame([{'quantity': w_sl.value, 'unit_price': w_gia.value,
            'fiscal_month': w_thang.value, 'region': w_vung.value,
            'group_name': w_nhom.value,
            'line_name': df['line_name'].mode()[0],
            'color': df['color'].mode()[0] if 'color' in df.columns else ''
        }])
        try:
            ket_qua = mo_hinh.predict(row)[0]
            mau = '#4CAF50' if ket_qua > df['line_total'].mean() else '#FF9800'
            display(HTML(f'''
<div style="background:{mau};color:white;padding:16px 24px;border-radius:10px;font-size:18px;margin:8px 0">
Doanh thu dự báo: <b>{ket_qua:.2f}</b> Triệu VNĐ</div>
<div style="color:#555;font-size:12px">Trung bình dataset: {df["line_total"].mean():.2f} Tr | Dự báo so TB: {((ket_qua/df["line_
'''))
        except Exception as e:
            print('Lỗi:', e)

for w in [w_sl, w_gia, w_nhom, w_vung, w_thang]:
    w.observe(du_bao, names='value')

display(widgets.VBox([w_sl, w_gia, w_nhom, w_vung, w_thang, out]))
du_bao(None)

In [ ]:

engine.dispose()
print('Đã đóng kết nối.')

```


Xe Đạp Thống Nhất — Phân Tích Chuỗi Thời Gian Doanh Số

Notebook 3: Mô Hình AR/ARMA

Huấn luyện mô hình AutoReg, tìm tham số p tối ưu, walk-forward validation. Theo cấu trúc WQU ADSL Dự Án 3.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import matplotlib.pyplot as plt
import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'iframe'
from sklearn.metrics import mean_absolute_error
from statsmodels.tsa.ar_model import AutoReg
from sqlalchemy import create_engine
```

1. Tải Dữ Liệu

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
def wrangle(engine):
    df = pd.read_sql_query(
        'SELECT order_date AS ngay, ROUND(SUM(line_total)/1000000.0,2) AS doanh_thu_ngay FROM tnbike.fact_sales WHERE order_date BETWEEN 2017-01-01 AND 2017-12-31',
        con=engine
    )
    df['ngay'] = pd.to_datetime(df['ngay'])
    df.set_index('ngay', inplace=True)
    y = df['doanh_thu_ngay'].resample('D').mean().ffill()
    return y

y = wrangle(engine)
nguong = int(len(y) * 0.9)
y_train = y.iloc[:nguong]
y_test = y.iloc[nguong:]
print('y_train:', y_train.shape, '| y_test:', y_test.shape)
```

2. Tìm Tham Số p Tối Ưu (Điều Chỉnh Mô Hình AR)

In []:

```
cac_p = range(1, 31)
danh_sach_mae = []
for p in cac_p:
    mo_hinh = AutoReg(y_train, lags=p).fit()
    y_du_doan = mo_hinh.predict().dropna()
    mae = mean_absolute_error(y_train.iloc[p:], y_du_doan)
    danh_sach_mae.append(mae)

mae_series = pd.Series(danh_sach_mae, name='mae', index=cac_p)
print(mae_series.head())
```

In []:

```
mae_series.plot(xlabel='Số Độ Trễ p', ylabel='MAE', title='MAE Mô Hình AR Theo Số Độ Trễ')
plt.tight_layout()
plt.show()
```

3. Mô Hình Tốt Nhất

In []:

```
p_tot_nhat = mae_series.idxmin()
print('Số độ trễ tốt nhất p =', p_tot_nhat)

mo_hinh_tot_nhat = AutoReg(y_train, lags=p_tot_nhat).fit()
phan_du = mo_hinh_tot_nhat.resid
```

```
phan_du.name = 'phần_dư'
phan_du.head()
```

4. Walk-Forward Validation (Kiểm Chứng Tiến Dần)

In []:

```
y_du_doan_wfv = pd.Series(dtype=float)
lich_su = y_train.copy()
for i in range(len(y_test)):
    m = AutoReg(lich_su, lags=p_tot_nhat).fit()
    du_doan_tiep = m.forecast()
    y_du_doan_wfv = pd.concat([y_du_doan_wfv, du_doan_tiep])
    lich_su = pd.concat([lich_su, y_test.iloc[[i]]])

y_du_doan_wfv.name = 'du_doan'
y_du_doan_wfv.index.name = 'ngay'
mae_wfv = mean_absolute_error(y_test, y_du_doan_wfv)
print('MAE Walk-Forward Validation:', f'{mae_wfv:,.0f} VNĐ')
```

5. Truyền Đạt Kết Quả — Biểu Đồ So Sánh

In []:

```
df_ket_qua = pd.DataFrame({'thuc_te': y_test, 'du_doan_wfv': y_du_doan_wfv})
fig = px.line(df_ket_qua, labels={'value': 'Doanh Thu (VNĐ)'})
fig.update_layout(
    title='Doanh Thu Ngày TNBike – Kiểm Chứng Walk-Forward',
    xaxis_title='Ngày',
    yaxis_title='Doanh Thu (VNĐ)'
)
fig.show()
```

6. DỰ BÁO DOANH SỐ Q2/2026 — Câu Hỏi 1

Yêu cầu đề thi:

- Tổng doanh số tháng 4, 5, 6/2026
- Phân tích theo nhóm 5 sản phẩm
- Top 20 mẫu xe dự kiến bán chạy nhất
- Cấp độ dự báo: theo tháng và theo tuần

In []:

```
# — Bước 1: Dự báo tổng doanh thu tháng 4-5-6/2026 —————
# Dùng mô hình AR tốt nhất đã huấn luyện, forecast 90 ngày tiếp theo
so_ngay_du_bao = 90 # Tháng 4 + 5 + 6/2026
```

```
du_bao_tuong_lai = mo_hinh_tot_nhat.forecast(steps=so_ngay_du_bao)
du_bao_tuong_lai.index = pd.date_range(
    start=y.index[-1] + pd.Timedelta(days=1),
    periods=so_ngay_du_bao,
    freq='D'
)
du_bao_tuong_lai.name = 'doanh_thu_du_bao'
```

```
# Lọc Q2/2026: tháng 4, 5, 6
q2_2026 = du_bao_tuong_lai[
    (du_bao_tuong_lai.index >= '2026-04-01') &
    (du_bao_tuong_lai.index <= '2026-06-30')
]
print('Số ngày dự báo trong Q2/2026:', len(q2_2026))
q2_2026.head()
```

In []:

```
# — Bước 2: Tổng doanh thu từng tháng Q2/2026 —————
du_bao_theo_thang = q2_2026.resample('ME').sum().rename('doanh_thu_du_bao')
du_bao_theo_thang.index = ['Tháng 4/2026', 'Tháng 5/2026', 'Tháng 6/2026']
```

```
print('=== TỔNG DOANH THU DỰ BÁO Q2/2026 ===')
for thang, dt in du_bao_theo_thang.items():
    print(f' {thang}: {dt:,.0f} VNĐ')
print(f' TỔNG Q2: {du_bao_theo_thang.sum():,.0f} VNĐ')
```

```
du_bao_theo_thang.plot(
    kind='bar', title='Dự Báo Doanh Thu Q2/2026 Theo Tháng',
    ylabel='Doanh Thu (VNĐ)', color='steelblue'
)
plt.tight_layout()
plt.show()
```

In []:

```
# — Bước 3: Dự báo theo tuần trong Q2/2026 —————
du_bao_theo_tuan = q2_2026.resample('W').sum().rename('doanh_thu_du_bao')
```

```

print('=== DỰ BÁO THEO TUẦN – Q2/2026 ===')
print(du_bao_theo_tuan.to_string())

fig = px.line(
    du_bao_theo_tuan.reset_index(),
    x='ngay', y='doanh thu du_bao',
    title='Dự Báo Doanh Thu Theo Tuần – Q2/2026',
    markers=True
)
fig.update_layout(xaxis_title='Tuần', yaxis_title='Doanh Thu (VNĐ)')
fig.show()

```

In []:

```

# — Bước 4: Dự báo theo nhóm 5 sản phẩm —————
# Xây AR model riêng cho từng nhóm sản phẩm
nhom_du_bao = {}

df_nhom = pd.read_sql_query(
    '''
    SELECT
    order_date,
    group_name,
    SUM(line_total) AS doanh_thu
    FROM tnbike.fact_sales
    WHERE order_date BETWEEN '2025-01-01' AND '2026-03-31'
    GROUP BY order_date, group_name
    ORDER BY order_date
    ''', con=engine
)
df_nhom['order_date'] = pd.to_datetime(df_nhom['order_date'])

for nhom in df_nhom['group_name'].dropna().unique():
    y_nhom = (
        df_nhom[df_nhom['group_name'] == nhom]
        .set_index('order_date')['doanh_thu']
        .resample('D').sum()
        .ffill()
    )
    if len(y_nhom) < 30:
        continue
    try:
        m = AutoReg(y_nhom, lags=min(7, len(y_nhom)//3)).fit()
        db = m.forecast(steps=90)
        db.index = pd.date_range(start=y_nhom.index[-1]+pd.Timedelta(days=1), periods=90, freq='D')
        q2 = db[(db.index >= '2026-04-01') & (db.index <= '2026-06-30')]
        nhom_du_bao[nhom] = q2.sum()
    except:
        pass

df_nhom_du_bao = pd.Series(nhom_du_bao, name='du_bao_q2').sort_values(ascending=False)
print('=== DỰ BÁO DOANH THU Q2/2026 THEO NHÓM SẢN PHẨM ===')
for nhom, dt in df_nhom_du_bao.items():
    print(f' {nhom}: {dt:,.1f} Triệu VNĐ')

df_nhom_du_bao.plot(kind='barh', title='Dự Báo Q2/2026 Theo Nhóm Sản Phẩm', xlabel='Doanh Thu (Triệu VNĐ)')
plt.tight_layout()
plt.show()

```

In []:

```

# — Bước 5: Top 20 mẫu xe dự kiến bán chạy nhất Q2/2026 ———
# Dùng xu hướng Q1/2026 làm nền dự báo (naive seasonal forecast)
df_top_sku = pd.read_sql_query(
    '''
    SELECT
    product_code,
    product_name,
    group_name,
    color,
    SUM(quantity) AS tong_so_luong,
    ROUND(SUM(line_total)/1000000.0,2) AS tong_doanh_thu,
    AVG(line_total) AS dt_tb
    FROM tnbike.fact_sales
    WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31'
    GROUP BY product_code, product_name, group_name, color
    ORDER BY tong_so_luong DESC
    LIMIT 20
    ''', con=engine
)

# Nhân hệ số mùa vụ Q2/Q1 từ lịch sử 2025
df_mua_vu = pd.read_sql_query(
    '''
    SELECT
    fiscal_quarter,
    SUM(line_total) AS doanh_thu
    FROM tnbike.fact_sales
    WHERE fiscal_year = 2025
    GROUP BY fiscal_quarter
    ORDER BY fiscal_quarter
    ''', con=engine
)

```

```

)
if len(df_mua_vu) >= 2:
    he_so_q2_q1 = df_mua_vu.set_index('fiscal_quarter')['doanh_thu'].get(2, 1) / df_mua_vu.set_index('fiscal_quarter')['doanh_thu'].get(1, 1)
else:
    he_so_q2_q1 = 1.0
print(f'Hệ số mùa vụ Q2/Q1 (2025): {he_so_q2_q1:.3f}')

df_top_sku['du_bao_q2_so_luong'] = (df_top_sku['tong_so_luong'] * he_so_q2_q1).round(0).astype(int)
df_top_sku['du_bao_q2_doanh_thu'] = (df_top_sku['tong_doanh_thu'] * he_so_q2_q1).round(0)

print()
print('=== TOP 20 MẪU XE DỰ KIẾN BÁN CHẠY NHẤT Q2/2026 ===')
print(df_top_sku[['product_name', 'group_name', 'color', 'du_bao_q2_so_luong', 'du_bao_q2_doanh_thu']].to_string(index=False))

In [ ]:

# Biểu đồ Top 20 SKU
fig = px.bar(
    df_top_sku.head(20),
    x='du_bao_q2_so_luong',
    y='product_name',
    color='group_name',
    orientation='h',
    title='Top 20 Mẫu Xe Dự Kiến Bán Chạy Nhất Q2/2026',
    labels={'du_bao_q2_so_luong': 'Số Lượng Dự Báo', 'product_name': 'Tên Sản Phẩm'})
fig.update_layout(yaxis={'categoryorder': 'total ascending'})
fig.show()

```

Phương Trình Mô Hình AR(p)

$$\hat{y}_t = \{\textcolor{royalblue}{\phi_1}\} \cdot \{\textcolor{gold}{y_{t-1}}\} + \{\textcolor{royalblue}{\phi_2}\} \cdot \{\textcolor{gold}{y_{t-2}}\} + \cdots + \{\textcolor{royalblue}{\phi_p}\} \cdot \{\textcolor{gold}{y_{t-p}}\} + \textcolor{violet}{\varepsilon_t}$$

ϕ (phi) = hệ số tự hồi quy | **$y(t-k)$** = doanh thu k ngày trước | **ε** = nhiễu ngẫu nhiên

In []:

```

from IPython.display import display, HTML
import numpy as np

# Hiển thị hệ số  $\phi$  của mô hình tốt nhất
try:
    params = mo_hinh_tot_nhat.params
    phi_html = '<table style="font-size:13px"><tr><th style="background:#f0f0f0;padding:4px 16px">Lag</th>'
    phi_html += '<th style="background:#f0f0f0;padding:4px 16px">Hệ Số  $\phi$ </th>'
    phi_html += '<th style="background:#f0f0f0;padding:4px 16px">Ý Nghĩa</th></tr>'
    for i, (k, v) in enumerate(params.items()):
        color = '#1565C0' if abs(v) > 0.1 else '#888'
        y_nghia = 'Ảnh hưởng mạnh' if abs(v) > 0.3 else ('Ảnh hưởng vừa' if abs(v) > 0.1 else 'Yếu')
        phi_html += f'<tr><td style="color:{color};font-weight:bold;padding:3px 16px">{k}</td>'
        phi_html += f'<td style="color:{color};font-weight:bold;text-align:right;padding:3px 16px">{v:+.4f}</td>'
        phi_html += f'<td style="color:#555;padding:3px 16px">{y_nghia}</td></tr>'
    phi_html += '</table>'
    display(HTML('<h4>Hệ Số  $\phi$  Mô Hình AR</h4>' + phi_html))
except Exception as e:
    print('Không lấy được params:', e)

```

In []:

```

# Chỉ tiêu hiệu quả mô hình AR
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

try:
    y_pred_ar = mo_hinh_tot_nhat.predict(start=len(y_train), end=len(y_train)+len(y_test)-1)
    mae_ar = mean_absolute_error(y_test, y_pred_ar)
    rmse_ar = np.sqrt(mean_squared_error(y_test, y_pred_ar))
    mape_ar = np.mean(np.abs((y_test.values - y_pred_ar.values) / (y_test.values + 1e-9))) * 100

    metrics_html = f'''
<div style="display:flex;gap:16px;flex-wrap:wrap;margin:12px 0">
<div style="background:#E3F2FD;padding:12px 20px;border-radius:8px;text-align:center">
<div style="font-size:11px;color:#555">MAE (Triệu VNĐ/ngày)</div>
<div style="font-size:22px;font-weight:bold;color:#1565C0">{mae_ar:.3f}</div>
</div>
<div style="background:#FFF9C4;padding:12px 20px;border-radius:8px;text-align:center">
<div style="font-size:11px;color:#555">RMSE</div>
<div style="font-size:22px;font-weight:bold;color:#F57F17">{rmse_ar:.3f}</div>
</div>
<div style="background:#FCE4EC;padding:12px 20px;border-radius:8px;text-align:center">
<div style="font-size:11px;color:#555">MAPE (%)</div>
<div style="font-size:22px;font-weight:bold;color:#B71C1C">{mape_ar:.1f}%</div>
</div>
'''
    from IPython.display import display, HTML
    display(HTML('<h4>Chỉ Tiêu Hiệu Quả Mô Hình AR</h4>' + metrics_html))

```

```
except Exception as e:
    print('Lỗi tính metrics:', e)
```

Công Cụ Dự Báo Doanh Thu (Non-Tech)[1](#)

Kéo thanh trượt để xem doanh thu dự báo theo khoảng thời gian

In []:

```
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
import matplotlib.pyplot as plt
import pandas as pd

w_steps = widgets.IntSlider(value=30, min=7, max=90, step=7,
                             description='Số ngày dự báo:', style={'description_width': '150px'}, layout=widgets.Layout(width='500px'))
w_nhom = widgets.Dropdown(
    options=['Tất cả'] + (df_nhom['group_name'].dropna().unique().tolist() if 'df_nhom' in dir() else []),
    description='Nhóm sản phẩm:', style={'description_width': '150px'})
out = widgets.Output()

def cap_nhat_du_bao(_):
    with out:
        clear_output(wait=True)
        steps = w_steps.value
        try:
            db = mo_hinh_tot_nhat.forecast(steps=steps)
            db.index = pd.date_range(start=y.index[-1] + pd.Timedelta(days=1), periods=steps, freq='D')
            db_monthly = db.resample('ME').sum()

            fig, axes = plt.subplots(1, 2, figsize=(14, 4))
            # Lịch sử + dự báo
            y.tail(60).plot(ax=axes[0], label='Thực tế', color='steelblue')
            db.plot(ax=axes[0], label='Dự báo', color='tomato', linestyle='--')
            axes[0].set_title(f'Dự Báo {steps} Ngày Tới')
            axes[0].set_ylabel('Triệu VND/ngày')
            axes[0].legend()
            # Tổng theo tháng
            db_monthly.plot(kind='bar', ax=axes[1], color='coral')
            axes[1].set_title('Tổng Doanh Thu Dự Báo Theo Tháng')
            axes[1].set_ylabel('Triệu VND')
            for bar in axes[1].patches:
                axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
                             f'{bar.get_height():.1f}', ha='center', va='bottom', fontsize=9)
            plt.tight_layout()
            plt.show()
            tong = db.sum()
            display(HTML(f'<div style="background:#E8F5E9;padding:10px 20px;border-radius:8px">'
                         f'Tổng dự báo <b>{steps} ngày</b>: <b style="color:#2E7D32;font-size:16px">{tong:.1f} Triệu VND</b></div>'))
        except Exception as e:
            print('Lỗi:', e)

w_steps.observe(cap_nhat_du_bao, names='value')
display(widgets.VBox([w_steps, out]))
cap_nhat_du_bao(None)
```

In []:

```
engine.dispose()
print('Đã đóng kết nối.')
```

Xe Đạp Thống Nhất — Phân Loại Đơn Hàng Giá Trị Cao

Notebook 2: Phân Loại

Hồi quy Logistic + Cây Quyết Định để phân loại đơn hàng giá trị cao. Theo cấu trúc WQU ADSL Dự Án 4.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from category_encoders import OrdinalEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay, classification_report
from sklearn.model_selection import train_test_split, validation_curve
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.impute import SimpleImputer
from sqlalchemy import create_engine
```

1. Kết Nối & Hàm wrangle()

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
def wrangle(engine):
    df = pd.read_sql_query(
        '''
        SELECT DISTINCT
        ol.line_id AS ma_dong,
        ol.quantity AS so_luong, ROUND(ol.unit_price/1000000.0,2) AS don_gia, ROUND(ol.line_total/1000000.0,2) AS doanh_thu,
        pr.color AS mau_sac,
        pl.line_name AS dong_xe,
        pg.group_name AS nhom_xe,
        p.region AS vung
        FROM tnbike.order_line ol
        JOIN tnbike.sales_order so ON ol.order_id = so.order_id
        JOIN tnbike.customer c ON so.customer_code = c.customer_code
        LEFT JOIN tnbike.province p ON c.province_id = p.province_id
        JOIN tnbike.product pr ON ol.product_code = pr.product_code
        LEFT JOIN tnbike.product_line pl ON pr.line_id = pl.line_id
        LEFT JOIN tnbike.product_group pg ON pl.group_code = pg.group_code
        WHERE so.order_date BETWEEN '2025-01-01' AND '2026-03-31'
        ''', con=engine, index_col='ma_dong'
    )
    # Tạo nhãn phân loại
    nguong = df['doanh_thu'].quantile(0.75)
    df['gia_tri_cao'] = (df['doanh_thu'] > nguong).astype(int)
    # Bỏ cột gây rò rỉ dữ liệu
    df.drop(columns=['doanh_thu'], inplace=True)
    return df

df = wrangle(engine)
print(df.shape)
df.head()
```

2. Tách Đặc Trưng và Nhãn (Dọc)

In []:

```
nhan = 'gia_tri_cao'
X = df.drop(columns=nhan)
y = df[nhan]
print('X shape:', X.shape)
```

3. Tách Tập Huấn Luyện / Kiểm Tra (Ngang)

In []:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print('X_train:', X_train.shape, '| X_test:', X_test.shape)
```

4. Mô Hình Cơ Sở (Baseline Accuracy)¶

```
In [ ]:
```

```
do_chinh_xac_cb = y_train.value_counts(normalize=True).max()
print('Độ chính xác cơ sở:', round(do_chinh_xac_cb, 4))
```

5. Hồi Quy Logistic¶

```
In [ ]:
```

```
clf_lr = make_pipeline(OrdinalEncoder(), SimpleImputer(), LogisticRegression(max_iter=1000, random_state=42))
clf_lr.fit(X_train, y_train)
print('LR – Tập Huấn Luyện:', round(clf_lr.score(X_train, y_train), 4))
print('LR – Tập Kiểm Tra :', round(clf_lr.score(X_test, y_test), 4))
```

6. Cây Quyết Định¶

```
In [ ]:
```

```
clf_dt = make_pipeline(OrdinalEncoder(), SimpleImputer(), DecisionTreeClassifier(max_depth=10, random_state=42))
clf_dt.fit(X_train, y_train)
print('DT – Tập Huấn Luyện:', round(clf_dt.score(X_train, y_train), 4))
print('DT – Tập Kiểm Tra :', round(clf_dt.score(X_test, y_test), 4))
```

7. Đường Cong Kiểm Chứng¶

```
In [ ]:
```

```
train_scores, val_scores = validation_curve(
    DecisionTreeClassifier(random_state=42),
    X_train.select_dtypes('number').fillna(0), y_train,
    param_name='max_depth', param_range=range(1, 20), cv=5, scoring='accuracy'
)
plt.plot(range(1,20), train_scores.mean(axis=1), label='Tập Huấn Luyện')
plt.plot(range(1,20), val_scores.mean(axis=1), label='Tập Kiểm Chứng')
plt.xlabel('max_depth')
plt.ylabel('Độ Chính Xác')
plt.title('Đường Cong Kiểm Chứng – Cây Quyết Định')
plt.legend()
plt.tight_layout()
plt.show()
```

8. Truyền Đạt Kết Quả – Ma Trận Nhầm Lẫn¶

```
In [ ]:
```

```
ConfusionMatrixDisplay.from_estimator(clf_dt, X_test, y_test)
plt.title('Ma Trận Nhầm Lẫn – Cây Quyết Định')
plt.show()
```

```
In [ ]:
```

```
print(classification_report(y_test, clf_dt.predict(X_test)))
```

```
In [ ]:
```

```
engine.dispose()
print('Đã đóng kết nối.')
```

Xe Đạp Thống Nhất — Phát Hiện Churn Đại Lý

Notebook 2: Mô Hình Random Forest

Random Forest + GridSearchCV + Xử lý mất cân bằng lớp. Theo cấu trúc WQU ADSL Dự Án 5.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import pickle
import matplotlib.pyplot as plt
import pandas as pd
from imblearn.over_sampling import RandomOverSampler
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sqlalchemy import create_engine
```

1. Kết Nối & Hàm wrangle()

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
def wrangle(engine):
    df = pd.read_sql_query(
        '''
        SELECT
        c.customer_code AS ma_khach,
        COUNT(DISTINCT so.so_number) AS so_don_hang,
        ROUND(COALESCE(SUM(ol.line_total),0)/1000000.0,2) AS tong_doanh_thu,
        ROUND(COALESCE(AVG(ol.line_total),0)/1000000.0,2) AS dt_tb_don,
        COALESCE(SUM(ol.quantity), 0) AS tong_so_luong,
        MAX(so.order_date) AS ngay_mua_cuoi
        FROM tnbike.customer c
        LEFT JOIN tnbike.sales_order so ON so.customer_code = c.customer_code
        LEFT JOIN tnbike.order_line ol ON ol.order_id = so.order_id
        GROUP BY c.customer_code
        ''', con=engine
    )
    df['ngay_mua_cuoi'] = pd.to_datetime(df['ngay_mua_cuoi'])
    moc = pd.Timestamp('2025-09-30')
    df['churn'] = ((df['ngay_mua_cuoi'] < moc) | df['ngay_mua_cuoi'].isna()).astype(int)
    df.drop(columns=['ma_khach', 'ngay_mua_cuoi'], inplace=True)
    return df

df = wrangle(engine)
print(df.shape)
df.head()
```

2. Ma Trận Đặc Trưng & Vector Nhãn

In []:

```
nhan = 'churn'
X = df.drop(columns=nhan)
y = df[nhan]
print('X shape:', X.shape)
```

3. Tách Tập Huấn Luyện / Kiểm Tra

In []:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print('X_train:', X_train.shape, 'X_test:', X_test.shape)
```

4. Lấy Mẫu Quá Mức (RandomOverSampler)

In []:


```
over_sampler = RandomOverSampler(random_state=42)
X_train_over, y_train_over = over_sampler.fit_resample(X_train, y_train)
print('X_train_over shape:', X_train_over.shape)
print('Phân bố sau oversampling:')
pd.Series(y_train_over).value_counts(normalize=True)
```

5. Kiểm Chứng Chéo (Cross-Validation)

In []:

```
clf = make_pipeline(SimpleImputer(), RandomForestClassifier(random_state=42))
cv_scores = cross_val_score(clf, X_train_over, y_train_over, cv=5, n_jobs=-1)
print('Điểm CV:', cv_scores)
print('Điểm CV trung bình:', cv_scores.mean().round(4))
```

6. Tìm Siêu Tham Số (GridSearchCV)

In []:

```
tham_so = {
    'simpleimputer__strategy': ['mean', 'median'],
    'randomforestclassifier__n_estimators': range(25, 100, 25),
    'randomforestclassifier__max_depth': range(10, 50, 10)
}
mo_hinh = GridSearchCV(clf, param_grid=tham_so, cv=5, n_jobs=-1, verbose=1)
mo_hinh.fit(X_train_over, y_train_over)
print('Tham số tốt nhất:', mo_hinh.best_params_)
```

7. Đánh Giá Mô Hình

In []:

```
print('Độ chính xác tập huấn luyện:', round(mo_hinh.score(X_train, y_train), 4))
print('Độ chính xác tập kiểm tra:', round(mo_hinh.score(X_test, y_test), 4))
```

8. Ma Trận Nhầm Lẫn

In []:

```
ConfusionMatrixDisplay.from_estimator(mo_hinh, X_test, y_test)
plt.title('Ma Trận Nhầm Lẫn - Random Forest')
plt.show()
```

9. Báo Cáo Phân Loại

In []:

```
print(classification_report(y_test, mo_hinh.predict(X_test)))
```

10. Tầm Quan Trọng Đặc Trưng

In []:

```
ten_dac_trung = X_train_over.columns
muc_do_quan_trong = mo_hinh.best_estimator_.named_steps['randomforestclassifier'].feature_importances_
qt = pd.Series(muc_do_quan_trong, index=ten_dac_trung).sort_values()
qt.tail(10).plot(kind='barh')
plt.xlabel('Mức Độ Quan Trọng')
plt.title('Top 10 Đặc Trưng Quan Trọng - Random Forest')
plt.tight_layout()
plt.show()
```

11. Lưu Mô Hình

In []:

```
with open('mo_hinh_churn.pkl', 'wb') as f:
    pickle.dump(mo_hinh, f)
print('Đã lưu mô hình.')
```

Câu Hỏi 3 (C.2): Dự Báo Xác Suất Đặt Hàng 30 Ngày & Ưu Tiên Tiếp Thị

Xác suất đại lý đặt hàng trong 30 ngày tới? Danh sách nguy cơ rời bỏ? Điểm xu hướng & mức độ ưu tiên tiếp thị?

In []:

```

# Câu hỏi 3a: Xác suất đặt hàng trong 30 ngày tới (predict_proba)
# fact_sales đã có sẵn line_total, customer_code, province_name, region
df_dai_ly = pd.read_sql_query(
    '''
    SELECT
    customer_code,
    customer_name,
    province_name,
    region,
    COUNT(DISTINCT so_number) AS so_don,
    ROUND(SUM(line_total)/1000000.0,2) AS tong_dt,
    ROUND(AVG(line_total)/1000000.0,2) AS trung_binh_don,
    MAX(order_date) AS don_cuoi,
    (CURRENT_DATE - MAX(order_date)) AS so_ngay_im,
    ROUND(STDDEV(line_total)/1000000.0,2) AS do_bien_dong
    FROM tnbike.fact_sales
    WHERE fiscal_year >= 2024
    GROUP BY customer_code, customer_name, province_name, region
    ''',
    engine
)
print('Đại lý:', df_dai_ly.shape)
df_dai_ly.head()

In [ ]:

# Chuẩn bị đặc trưng cho dự báo
X_du_bao = df_dai_ly[['so_don', 'tong_dt', 'trung_binh_don', 'so_ngay_im', 'do_bien_dong']].copy()
X_du_bao = X_du_bao.fillna(0)

# Xác suất churn (class=1: có nguy cơ rời bỏ)
xac_suat_churn = mo_hinh.predict_proba(X_du_bao)[:, 1]

# Xác suất đặt hàng 30 ngày = 1 - xác suất churn
xac_suat_dat_hang = 1 - xac_suat_churn

df_du_bao = df_dai_ly[['customer_code', 'customer_name', 'province_name', 'region', 'so_ngay_im', 'don_cuoi']].copy()
df_du_bao['xac_suat_churn'] = xac_suat_churn.round(4)
df_du_bao['xac_suat_dat_hang'] = xac_suat_dat_hang.round(4)
print(df_du_bao.head())

In [ ]:

# Câu hỏi 3b: Danh sách đại lý có nguy cơ rời bỏ (xác suất churn cao nhất)
nguong_nguy_co = 0.6 # Ngưỡng nguy cơ cao

df_nguy_co = (
    df_du_bao[df_du_bao['xac_suat_churn'] >= nguong_nguy_co]
    .sort_values('xac_suat_churn', ascending=False)
    .reset_index(drop=True)
)
df_nguy_co.index += 1 # Bắt đầu từ 1

print(f'=== Danh Sách {len(df_nguy_co)} Đại Lý Có Nguy Cơ Rời Bỏ (≥{nguong_nguy_co*100:.0f}%) ===')
print(df_nguy_co[['customer_name', 'province_name', 'region', 'so_ngay_im', 'xac_suat_churn', 'xac_suat_dat_hang']].to_string())

In [ ]:

# Câu hỏi 3c: Điểm xu hướng mua hàng & mức độ ưu tiên tiếp thị
import numpy as np

def tinh_diem_xu_huong(row):
    """Điểm xu hướng: kết hợp tần suất, giá trị, thời gian im lặng."""
    diem_im_lang = max(0, 100 - float(row['so_ngay_im']) * 2) # Trừ điểm mỗi ngày im
    diem_tan_suat = min(100, float(row.get('so_don', 0)) * 5) # Cộng điểm theo số đơn
    diem_gia_tri = min(100, float(row.get('tong_dt', 0)) / 1e7) # Doanh thu chuẩn hoá
    return round((diem_im_lang * 0.4 + diem_tan_suat * 0.3 + diem_gia_tri * 0.3), 2)

df_uu_tien = df_du_bao.copy()
df_uu_tien['so_don'] = df_dai_ly['so_don'].values
df_uu_tien['tong_dt'] = df_dai_ly['tong_dt'].values
df_uu_tien['diem_xu_huong'] = df_uu_tien.apply(tinh_diem_xu_huong, axis=1)

def xep_loai_uu_tien(row):
    if row['xac_suat_churn'] >= 0.7:
        return 'Khẩn Cấp'
    elif row['xac_suat_churn'] >= 0.5:
        return 'Theo Dõi'
    else:
        return 'Ổn Định'

df_uu_tien['uu_tien_tiep_thi'] = df_uu_tien.apply(xep_loai_uu_tien, axis=1)
df_uu_tien_sx = df_uu_tien.sort_values(['xac_suat_churn', 'diem_xu_huong'], ascending=[False, True])

print('=== Bảng Ưu Tiên Tiếp Thị ===')
print(df_uu_tien_sx[['customer_name', 'region', 'xac_suat_churn', 'diem_xu_huong', 'uu_tien_tiep_thi']].head(20).to_string())

In [ ]:

# Biểu đồ phân bố nguy cơ churn theo miền
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

```

```
# Histogram xác suất churn
axes[0].hist(df_du_bao['xac_suat_churn'], bins=20, color='tomato', edgecolor='white')
axes[0].axvline(x=0.6, color='darkred', linestyle='--', label='Ngưỡng 60%')
axes[0].set_title('Phân Bó Xác Suất Churn')
axes[0].set_xlabel('Xác suất churn')
axes[0].set_ylabel('Số đại lý')
axes[0].legend()

# Phân bố ưu tiên tiếp thị theo miền
uu_tien_mien = df_uu_tien.groupby(['region', 'uu_tien_tiep_thi']).size().unstack(fill_value=0)
uu_tien_mien.plot(kind='bar', ax=axes[1], colormap='RdYlGn')
axes[1].set_title('Đại Lý Theo Mức Ưu Tiên Tiếp Thị & Miền')
axes[1].set_xlabel('Miền')
axes[1].set_ylabel('Số đại lý')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
print(f'\nTổng đại lý khẩn cấp (≥70%): {len(df_uu_tien[df_uu_tien["xac_suat_churn"]>=0.7])}')
print(f'\nTổng đại lý theo dõi (50-70%): {len(df_uu_tien[(df_uu_tien["xac_suat_churn"]>=0.5) & (df_uu_tien["xac_suat_churn"]<0.7)])}')
```

Phương Trình Random Forest

$$\hat{P}(\text{churn}) = \frac{1}{T} \sum_{t=1}^T \{h_t\} \{X\}$$

$$\text{Quan trọng} \{X_j\} = \sum_{t=1}^T \sum_{n \in h_t} \{\Delta I(n, X_j)\}$$

h(t) = cây quyết định thứ t | **X** = đặc trưng đại lý | **T** = tổng số cây **P(churn)** = xác suất đại lý rời bỏ trong 30 ngày tới

In []:

```
# Chỉ tiêu hiệu quả mô hình Random Forest
from sklearn.metrics import (classification_report, roc_auc_score,
                             precision_score, recall_score, f1_score, accuracy_score)
from IPython.display import display, HTML

y_pred = mo_hinh.predict(X_test)
y_proba = mo_hinh.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
auc = roc_auc_score(y_test, y_proba)

metrics_html = f'''
<div style="display:flex;gap:14px;flex-wrap:wrap;margin:12px 0">
  <div style="background:#E3F2FD;padding:12px 18px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">Accuracy</div>
    <div style="font-size:22px;font-weight:bold;color:#1565C0">{acc:.1%}</div>
  </div>
  <div style="background:#FFF9C4;padding:12px 18px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">Precision</div>
    <div style="font-size:22px;font-weight:bold;color:#F57F17">{prec:.1%}</div>
  </div>
  <div style="background:#E8F5E9;padding:12px 18px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">Recall</div>
    <div style="font-size:22px;font-weight:bold;color:#2E7D32">{rec:.1%}</div>
  </div>
  <div style="background:#F3E5F5;padding:12px 18px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">F1-Score</div>
    <div style="font-size:22px;font-weight:bold;color:#6A1B9A">{f1:.1%}</div>
  </div>
  <div style="background:#FCE4EC;padding:12px 18px;border-radius:8px;text-align:center">
    <div style="font-size:11px;color:#555">AUC-ROC</div>
    <div style="font-size:22px;font-weight:bold;color:#B71C1C">{auc:.4f}</div>
  </div>
</div>'''
display(HTML('<h4>Chỉ Tiêu Hiệu Quả Mô Hình Random Forest</h4>' + metrics_html))
```

Công Cụ Tra Cứu Nguy Cơ Churn Đại Lý (Non-Tech)

Nhập mã hoặc tên đại lý → hệ thống cho biết xác suất churn & mức ưu tiên tiếp thị

In []:

```
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

w_search = widgets.Text(placeholder='Nhập tên hoặc mã đại lý...', description='Tìm đại lý:',
                        style={'description_width': '100px'}, layout=widgets.Layout(width='400px'))
w_nguong = widgets.FloatSlider(value=0.5, min=0.1, max=0.9, step=0.05,
                               description='Ngưỡng cảnh báo:', style={'description_width': '140px'},
```

```
layout=widgets.Layout(width='450px'),
readout_format='.0%')
out = widgets.Output()

def tim_dai_ly(_):
    with out:
        clear_output(wait=True)
        tu_khoa = w_search.value.strip().lower()
        if not tu_khoa:
            # Hiển thị top 10 nguy cơ cao nhất
            top10 = df_uu_tien.nlargest(10, 'xac_suat_churn')[['customer_name','region','so_ngay_im','xac_suat_churn','uu_tien_tiep_thi']]
            display(HTML('<b>Top 10 đại lý nguy cơ cao nhất:</b>'))
            display(top10)
            return

        ket_qua = df_uu_tien[
            df_uu_tien['customer_name'].str.lower().str.contains(tu_khoa, na=False) |
            df_uu_tien['customer_code'].str.lower().str.contains(tu_khoa, na=False)
        ]

        if ket_qua.empty:
            display(HTML('<div style="color:#F44336">Không tìm thấy đại lý phù hợp.</div>'))
            return

        for _, row in ket_qua.head(5).iterrows():
            p = row['xac_suat_churn']
            if p >= 0.7: bg, icon = '#FFEBEE', ''
            elif p >= 0.5: bg, icon = '#FFF9C4', ''
            else: bg, icon = '#E8F5E9', ''
            canh_bao = ' CẦN GỌI NGAY' if p >= w_nguong.value else 'Theo dõi thêm'
            display(HTML(f'''
<div style="background:{bg};padding:12px 16px;border-radius:8px;margin:6px 0">
<b style="font-size:15px">{icon} {row["customer_name"]}</b>
<span style="color:#888;font-size:12px"> – {row.get("region","")} | Im lặn {row["so_ngay_im"]} ngày</span><br>
<span>Xác suất churn: <b style="font-size:16px;color:#D32F2F">{p:.0%}</b></span>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
<span>Xác suất đặt hàng 30 ngày: <b style="color:#2E7D32">{row["xac_suat_dat_hang"]:.0%}</b></span>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~<b style="color:#E65100">{canh_bao}</b>
</div>
'''))

w_search.observe(tim_dai_ly, names='value')
w_nguong.observe(tim_dai_ly, names='value')
display(widgets.VBox([
    widgets.HTML('<h4> Tra Cứu Nguy Cơ Churn Đại Lý</h4>'),
    w_search, w_nguong, out
]))
tim_dai_ly(None)
```

```
In []:
```

```
engine.dispose()
print('Đã đóng kết nối.')
```

Xe Đạp Thống Nhất — Phân Nhóm Khách Hàng RFM

Notebook 3: Dashboard Tương Tác

Dashboard phân tích cụm khách hàng. Theo cấu trúc WQU ADSL Dự Án 6.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'iframe'
import ipywidgets as widgets
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sqlalchemy import create_engine
```

1. Tải Dữ Liệu & Phân Nhóm

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
df = pd.read_sql_query(
    """
    SELECT c.customer_code AS ma_khach,
           p.region AS vung,
           COUNT(DISTINCT so.so_number) AS so_don_hang,
           ROUND(COALESCE(SUM(ol.line_total),0)/1000000.0,2) AS tong_doanh_thu,
           ROUND(COALESCE(AVG(ol.line_total),0)/1000000.0,2) AS dt_tb_don,
           COALESCE(SUM(ol.quantity), 0) AS tong_so_luong
    FROM tnbike.customer c
    LEFT JOIN tnbike.sales_order so ON so.customer_code = c.customer_code
    LEFT JOIN tnbike.order_line ol ON ol.order_id = so.order_id
    LEFT JOIN tnbike.province p ON p.province_id = c.province_id
    GROUP BY c.customer_code, p.region
    """, con=engine
)
df.set_index('ma_khach', inplace=True)
df.fillna(0, inplace=True)

X = df[['so_don_hang', 'tong_doanh_thu', 'dt_tb_don', 'tong_so_luong']]
mo_hinh = make_pipeline(StandardScaler(), KMeans(n_clusters=3, random_state=42, n_init=10))
mo_hinh.fit(X)
df['cum'] = mo_hinh.named_steps['kmeans'].labels_
print('Đã phân nhóm xong.')
```

2. Tương Tác: Lọc Theo Cụm

In []:

```
def hien_thi_cum(cum_id):
    tap_con = df[df['cum'] == cum_id]
    print(f'Cụm {cum_id}: {len(tap_con)} khách hàng')
    display(tap_con.describe())

widget_cum = widgets.IntSlider(min=0, max=2, value=0, description='Cụm:')
interact(hien_thi_cum, cum_id=widget_cum)
```

3. Phân Bố Vùng Theo Cụm

In []:

```
fig = px.histogram(df.reset_index(), x='vung', color=df['cum'].astype(str),
    barmode='group', title='Phân Bố Vùng Theo Cụm Khách Hàng')
fig.update_layout(xaxis_title='Vùng', yaxis_title='Số Khách Hàng')
fig.show()
```

4. Tán Xạ PCA

In []:

```
pca = PCA(n_components=2, random_state=42)
X_pca = pd.DataFrame(pca.fit_transform(X), columns=['PC1', 'PC2'])
fig = px.scatter(X_pca, x='PC1', y='PC2', color=df['cum'].astype(str),
                 title='PCA: Phân Nhóm Khách Hàng TNBike')
fig.show()
```

5. Doanh Thu Trung Bình Theo Cụm

In []:

```
tb_cum = df.groupby('cum')[['so_don_hang', 'tong_doanh_thu', 'dt_tb_don', 'tong_so_luong']].mean()
fig = px.bar(tb_cum, barmode='group', title='Đặc Trưng Trung Bình Theo Cụm')
fig.update_layout(xaxis_title='Cụm', yaxis_title='Giá Trị Trung Bình')
fig.show()
```

In []:

```
engine.dispose()
print('Đã đóng kết nối.')
```

Xe Đạp Thống Nhất — AB Testing Chiến Dịch Bán Hàng

Notebook 3: Phân Tích AB Testing

Kiểm định Chi-Square + Sức mạnh thống kê cho chiến dịch khuyến mãi. Theo cấu trúc WQU ADSL Dự Án 7.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import math
import scipy
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'iframe'
from statsmodels.stats.contingency_tables import Table2x2
from statsmodels.stats.power import GofChiSquarePower
from sqlalchemy import create_engine
```

1. Tải Dữ Liệu Thử Nghiệm

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
df = pd.read_sql_query(
    """
    SELECT
    so.so_number,
    so.customer_code,
    so.order_date,
    so.fiscal_quarter AS quy,
    CASE
    WHEN so.fiscal_quarter >= 3 THEN 'có_khuyến_mãi (thử_nghiệm)'
    ELSE 'không_khuyến_mãi (đối_chứng)'
    END AS nhom,
    CASE
    WHEN so.total_amount > 5000000 THEN 'chuyển_đổi'
    ELSE 'không_chuyển_đổi'
    END AS ket_qua
    FROM tnbike.sales_order so
    WHERE so.order_date BETWEEN '2025-04-01' AND '2025-12-31'
    ''', con=engine
)
print('Kích thước:', df.shape)
df.head()
```

2. Bảng Chéo (Contingency Table)

In []:

```
bang_cheo = pd.crosstab(
    index=df['nhom'],
    columns=df['ket_qua'],
    normalize=False
)
bang_cheo
```

3. Biểu Đồ Thanh — Kết Quả Chuyển Đổi

In []:

```
def ve_bieu_do_thanh():
    fig = px.bar(
        data_frame=bang_cheo,
        barmode='group',
        title='Tỷ Lệ Chuyển Đổi: Đối Chứng vs Thử Nghiệm'
    )
    fig.update_layout(xaxis_title='Nhóm', yaxis_title='Số Lượng')
    return fig
```

```
bieu_do = ve_bieu_do_thanh()
bieu_do.show()
```

4. Kiểm Định Chi-Square

In []:

```
bang_lien_ket = Table2x2(bang_cheo.values)
kiem_dinh_chi2 = bang_lien_ket.test_nominal_association()
ti_le_odds = bang_lien_ket.oddsratio.round(1)

print('Thống kê Chi-Square:', round(kiem_dinh_chi2.statistic, 4))
print('Giá trị p:', round(kiem_dinh_chi2.pvalue, 4))
print('Tỷ lệ odds:', ti_le_odds)
```

5. Sức Mạnh Thống Kê (Statistical Power)

In []:

```
suc_manh_chi2 = GofChisquarePower()
co_mau_nhom = math.ceil(suc_manh_chi2.solve_power(
    effect_size=0.5,
    alpha=0.05,
    power=0.8
))
print('Cỡ mẫu tối thiểu mỗi nhóm:', co_mau_nhom)
```

6. Tỷ Lệ Chuyển Đổi Theo Nhóm

In []:

```
ty_le_chuyen_doi = df.groupby('nhom')['ket_qua'].apply(
    lambda x: (x == 'chuyen_doi').mean()
)
print('Tỷ lệ chuyển đổi theo nhóm:')
print(ty_le_chuyen_doi)
```

7. Trực Quan Hoá Phân Bố Đơn Hàng Theo Tỉnh/Thành

In []:

```
df_tinh = pd.read_sql_query(
    '''
    SELECT p.province_name AS tinh_thanh, COUNT(so.so_number) AS so_don_hang
    FROM tnbike.sales_order so
    JOIN tnbike.customer c ON c.customer_code = so.customer_code
    LEFT JOIN tnbike.province p ON p.province_id = c.province_id
    WHERE so.order_date BETWEEN '2025-04-01' AND '2025-12-31'
    GROUP BY p.province_name
    ORDER BY so_don_hang DESC
    LIMIT 20
    ''', con=engine
)
fig = px.bar(df_tinh, x='tinh_thanh', y='so_don_hang',
    title='Số Đơn Hàng Theo Tỉnh/Thành (Top 20)')
fig.update_layout(xaxis_title='Tỉnh/Thành', yaxis_title='Số Đơn Hàng')
fig.show()
```

In []:

```
engine.dispose()
print('Đã đóng kết nối.')
```


Xe Đạp Thống Nhất — Dự Báo Biến Động Doanh Thu GARCH

Notebook 4: Dự Báo

Tạo dự báo biến động và truyền đạt kết quả. Theo cấu trúc WQU ADSL Dự Án 8.

In []:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import joblib
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'iframe'
from arch import arch_model
from sqlalchemy import create_engine
```

1. Tải Dữ Liệu

In []:

```
engine = create_engine('postgresql://neondb_owner:npg_kyw0DxEUvMK7@ep-little-fog-aprabc8e.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require')
```

In []:

```
df = pd.read_sql_query(
    '''
    SELECT order_date AS date, ROUND(SUM(line_total)/1000000.0,2) AS daily_revenue
    FROM tnbike.fact_sales
    WHERE order_date BETWEEN '2025-01-01' AND '2026-03-31'
    GROUP BY order_date ORDER BY order_date
    ''', con=engine, parse_dates=['date'], index_col='date'
)
df['loi_suat'] = df['daily_revenue'].pct_change() * 100
df.dropna(inplace=True)
y = df['loi_suat']
nguong = int(len(y) * 0.9)
y_train = y.iloc[:nguong]
y_test = y.iloc[nguong:]
print('Dữ liệu đã tải. y_train:', y_train.shape)
```

2. Huấn Luyện Mô Hình GARCH Cuối Cùng

In []:

```
mo_hinh_cuoi = arch_model(y_train, p=1, q=1, rescale=False).fit(dispatch=0)
print('AIC:', round(mo_hinh_cuoi.aic, 4))
print('BIC:', round(mo_hinh_cuoi.bic, 4))
```

3. Dự Báo Biến Động (5 ngày làm việc tiếp theo)

In []:

```
def du_bao_bien_dong(mo_hinh, horizon=5):
    du_bao = mo_hinh.forecast(horizon=horizon, reindex=False).variance
    ngay_bat_dau = du_bao.index[0] + pd.DateOffset(days=1)
    ngay_du_bao = pd.bdate_range(start=ngay_bat_dau, periods=du_bao.shape[1])
    du_lieu_db = du_bao.values.flatten() ** 0.5
    return pd.Series(du_lieu_db, index=[d.isoformat() for d in ngay_du_bao])

du_bao_bien_dong_5n = du_bao_bien_dong(mo_hinh_cuoi, horizon=5)
print('Dự báo biến động 5 ngày làm việc tiếp theo:')
print(du_bao_bien_dong_5n)
```

4. Walk-Forward Validation — Biến Động

In []:

```
du_doan_wfv = []
lich_su = y_train.copy()
for i in range(min(len(y_test), 30)):
```

```

m = arch_model(lich_su, p=1, q=1, rescale=False).fit(dispatch=0)
du_bao_ln = m.forecast(horizon=1, reindex=False).variance.values.flatten()[0] ** 0.5
du_doan_wfv.append(du_bao_ln)
lich_su = pd.concat([lich_su, y_test.iloc[[i]]])

du_doan_wfv = pd.Series(du_doan_wfv, index=y_test.index[:len(du_doan_wfv)])
print('Walk-forward validation hoàn thành.')
du_doan_wfv.head()

```

5. Truyền Đạt Kết Quả — Biểu Đồ So Sánh

In []:

```

df_bieu_do = pd.DataFrame({
    'loi_suat_thuc': y_test.iloc[:len(du_doan_wfv)],
    'bien_dong_du_bao': du_doan_wfv
})

fig = px.line(df_bieu_do, title='Lợi Suất Thực Tế vs Biến Động Dự Báo - TNBike')
fig.update_layout(xaxis_title='Ngày', yaxis_title='Giá Trị (%)')
fig.show()

```

6. Lưu Mô Hình

In []:

```

joblib.dump(mo_hinh_cuoi, 'mo_hinh_garch_tnbike.pkl')
print('Đã lưu mô hình GARCH.')

```

In []:

```

engine.dispose()
print('Đã đóng kết nối.')

```